TBWorldview 1.0.0r1

Table of Contents

Formatted for double-sided printing. Each major section starts on a recto (odd) page so the printout can be filed into a binder. A section that ends on an odd page is followed by a blank page. Print duplex, flip on the long edge.

Sections through TB80NET are the v1.0.0 print, unchanged. 1.0.0r1 adds the ledger, the environment notes, the audience notes, the 11.5 export, and the module catalog.

ToolBook Directory — 3

Overview — 5

Architecture — 9

Naming Progress — 11

Book Format — 13

OpenScript — 15

SoftICE Notes — 17

Runtimes — 19

TB80BAS Known Names — 21

TB80BMP Known Names — 23

TB80CMP Known Names — 25

TB80RTM Known Names — 27

TB80UTL Known Names — 29

TB80W16 Known Names — 31

TB80LNL Known Names — 33

TB80MM Known Names — 35

TB80RCR Known Names — 37

TB80FLT Known Names — 39

TB80NET Known Names — 41

Ledger — 43

Win16, Windows 98, Windows XP — 49

Who used ToolBook — 51

ToolBook 11.5 and TBKX — 53

Module catalog — 55

Original ToolBook Directory Contents

Files that belong in the Nida ATT ToolBook installation

Source of truth

The file **toolbook.lst** is the runtime load list. It names every book, DLL, and executable the Nida shell expects. **version.ini** reports 03.001.0006 dated 04-18-05. Modern course-menu rebuilds (COUR1.EXE, courses_menu.exe, and their MinGW runtimes) are not in this list and were not part of the original Nida ToolBook directory.

Nida system books and support

NC81BASE.SBK, NC81RECS.SBK, NC81TEST.SBK, NC81TRNR.SBK, NC81BANK.SBK
NC81INT.DLL, NC81REC.DLL
NIDA81.TBK
nida.ttf (font)
CAILT.DAT, STDAPPS.LST, version.ini

Core ToolBook 8 binaries

TB80RTM.EXE (engine), TB80RUN.EXE, TB80MAIN.EXE, Tbload.exe
TB80BAS.DLL, TB80BMP.DLL, TB80CMP.DLL, TB80UTL.DLL
TB80W16.DLL, TB80W32.DLL, TB80ENU1.DLL, TB80ENU2.DLL
TB80OLE.DLL, TB80VBX.DLL, TB80OCX.DLL, TB80ACX.DLL
TB80FLT.DLL, TB80LNL.DLL, TB80MM.DLL, TB80NET.EXE
TB80RCR.DLL, TB80RMT.DLL, TB80JPG.DLL, TB80F32L.DLL
TBCBT.DLL, TBDB3.DLL, TBDLG.DLL, TBDOS.DLL
TBFILE32.DLL, TBPDX.DLL, TBREG.DLL, TBWIN.DLL

System books

TB80R.SBK, TB80ANM.SBK, TB80HYP.SBK

Not part of the original directory

COUR1.EXE / COUR1.BAS, COURMENU.*, COURMENV.*, thecourse*.bas
courses_menu.exe / courses_menu.bas and the CSV exports
libgcc_s_seh-1.dll, libstdc++-6.dll, libwinpthread-1.dll (MinGW runtimes for the modern menu)
courses/ subdirectory, runtime2/, courses.xml (menu export)
zlib1.dll (not listed in toolbook.lst)

ToolBook 8 Reverse Engineering

Overview & Status Report

October 2026 — Nida ATT Master_3 Courses

1. What These Books Are

The course files are classic Asymetrix/Click2learn ToolBook books (ToolBook II 8.1 shells, book version 01 00 00 08). Each **.tbk** begins with a short PE stub (0xC00 bytes, no real code) followed by the book body. The magic **ZZZTBKBOOKLOC** and the value **0x22222222** mark the start of the real book data. The opener is **tbload.exe** via **MINSTB32.dll**.

The runtime engine that actually interprets the book is the 16-bit NE binary **TB80RTM.EXE** (unpacked from its SLR Optloader wrapper). Supporting DLLs (**TB80BAS**, **TB80CMP**, **TB80BMP**, **TB80UTL**, **TB80W16**, and others) supply object types, painting, OpenScript compilation/expansion, bitmaps, and windowing.

The courses themselves are Navy electronics CAI. **courses.xml** is a menu export that lists 0_MASTER3 plus twenty rating tracks. On disk there are hundreds of lesson **.tbk** files. Work has concentrated on **9e954d01.tbk** (Logic Families) plus the shared shell **nida81.tbk** and the system books.

2. On-Disk Structure (Understood)

Page objects are the only records that begin with the marker **80 65 00 05 00 02 02**. Inside each page the widgets are nested chunks that begin **14 00 00**. A typical property block starts **14 00 00 01**. Field text is stored as a Text property (length-prefixed, often with a leading CRLF).

Standard property IDs (bounds = 0x4004, text = 0x402e, script = 0x4023, vertices = 0x4034, etc.) were recovered from NE resource type 0x2d1 / id 0x2d0 in TB80BAS.

Object-type names (Page, Button, Field, Picture, Polygon, Arc, Curve, Rectangle, etc.) live in a classification table inside TB80UTL.DLL and are also present as resources in the system .sbk files.

3. What Modern Readers Already Do

Several generations of clean-room readers exist:

• Early HTML/XML viewers (viewer, viewer2, viewer3) that walk the page marker, extract text and quoted jumps.
• Maybe2 / Maybe4 lessonshells: JSON with pages, Next/Back, quiz choices, and decoded script.
• runtime/ and runtime_cpp/: Python and C++ packages that parse a book and emit HTML or drive a simple player.
• Brickwall / Foolbook: self-contained GUI readers that show page lists, field text, and quoted jumps.

All of these can already display the textual content, navigation, and quizzes of books that resemble Logic Families. They do not yet paint arbitrary polygons, arcs, curves, or placed bitmaps for the full corpus.

4. Function Naming Status

Pattern-based naming across decompiled modules has identified ~1,918 nameable functions (~17.5% of analyzed). Manual confirmation has produced over 100 named functions, concentrated in paint, scale, property, memory, and string helpers.

See **Modules/** subdirectories for per-module confirmed names and nameable lists. See **03_Naming_Progress.pdf** for the full summary.

5. Open Work

• Bounds extractor (record-to-object+0xF) still missing from decompiles.
• Full OpenScript execution and scoring.
• Complete vector painting (arcs, curves, computed brackets) and bitmap placement.
• NE host that fully opens a book under emulation.

Document Map

This package is organized for printing:

Main directory — overarching documentation (this overview, architecture, naming progress).
Modules/tb80xxx_knownnames/ — confirmed function names and nameable lists per DLL.
BookFormat/ — on-disk record layouts, polygon/arc geometry, property IDs.
OpenScript/ — bytecode decoder notes and extracted handlers.
SoftICE/ — dynamic analysis notes and live object layouts.
Runtimes/ — clean-room reader status.

ToolBook 8 Architecture

Runtime flow, modules, and object model

High-Level Architecture

ToolBook is an interpreted, object-oriented, event-driven system built around a book metaphor. A .tbk file contains a PE stub, ZZZTBKBOOKLOC magic, and a JBO body of page records. TB80RTM.EXE is the engine: book loader, page table, message dispatch, and window classes (TBKMain, TBKClient, TBKChildWindow, SchedulerWndTB80). Supporting DLLs provide objects, painting, OpenScript, bitmaps, and type tables.

Key Modules (16-bit NE unless noted)

Module	Role
TB80RTM.EXE	Main engine, book open, windowing, message loop
TB80BAS.DLL	Objects, properties, painters, scale twips, type dispatch
TB80CMP.DLL	OpenScript bytecode, keyword pool, expand/emit
TB80BMP.DLL	Bitmaps, DIB, palette, graphic FX, LZ77
TB80UTL.DLL	Object type table (Page, Polygon, Field, …), helpers
TB80W16.DLL	Thunk layer, graphic IDs, script editor commands
nida81.tbk + .sbk	Shared shell, system books (base, recs, test, trnr, tb80r)

Book Open Path (partial)

tbload finds ZZZTBKBOOKLOC → JBO signature checked by virtual method → page structures allocated → CDBSENDCREATE with bounds → FUN_12c8_0850 creates object and writes bounds to object+0xF. The record-to-bounds extractor itself has not yet been located in the decompiles.

Painting Path

Live objects carry bounds at +0xF. Polygon painter (TB80BAS seg 0x82) reads count at +0x30 and point array at +0x32, scales twips via ScaleTwipsToScreen ((stored*96+720)/1440), then calls GDI!POLYGON. Arc painter (seg 0x7B) uses four twip pairs; rectangle painter reads bounds and calls GDI!RECTANGLE. Many shapes are computed from object bounds rather than stored point lists.

OpenScript

Handlers live in page records. Keyword tables are NE resources 0x77/0x78 in TB80CMP. Decoder recovers readable source (“go to page "P400"”, “send next”) including 0xC0–0xFF back-references from the compressed input stream.

Function Naming Progress

Pattern-based nameable + confirmed renames (2026-10-11)

Overall Coverage

Modules analyzed: 8 of 13. Unique functions (analyzed): ~10,991. Nameable by pattern: ~1,918 (17.5%). 30% target would be ~3,297; gap ~1,379. Confirmed (manual) names exceed 100 and are concentrated in paint, scale, property, memory, and string helpers.

Per-Module Nameable

Module	Unique	Nameable	%
TB80BAS	3,739	650	17.4%
TB80RTM	2,669	873	32.7%
TB80CMP	1,058	59	5.6%
TB80BMP	381	99	26.0%
TB80UTL	192	79	41.1%
TB80W16	330	57	17.3%
TB80LNL	805	98	12.2%

Confirmed Names by Category (selected)

Paint/Draw: PaintPolygon, PaintPolyline, PaintArc, PaintEllipse, PaintRoundRect, PaintRectangle, BitBlt helpers, CreateDibBitmap, Lz77Expand.
Scale/Property: ScaleTwipsToScreen, SetProperty, SetPointArray, SetupPaintRect, SetMultiProperty, ValidateProperty.
Memory/String: Alloc/Lock/StrCpy/StrLen/StrCat helpers across modules.
Compiler: LoadCompilerResource, ExpandTextCore, CompilerStr*.

Notes

RTM is high because it is the main engine with heavy GDI, message, and property dispatch. CMP is low because it is mostly compiler emit routines and resource tables. Confirmed names are the ones manually verified and present in the rename_*.py Ghidra scripts. See Modules/ subdirectories for full confirmed lists and nameable TSVs.

Book File Format

On-disk records, properties, and geometry

File Header

Each .tbk begins with a 0xC00-byte PE stub containing no executable code. The real book starts at the ZZZTBKBOOKLOC marker (file offset 0xC00 + magic 0x22222222). Book signature is typically 03 4A 42 4F 01 00 00 08 (JBO + version).

Page Records

Page objects are the only records that begin with the marker **80 65 00 05 00 02 02**. Name is a 32-byte field. Property block at +43 starts 14 00 00 01. Widgets are nested 14 00 00 chunks inside the page.

Property IDs (from TB80BAS resource)

ID	Name	Notes
0x4004	bounds	Object bounds (twips)
0x402e	text	Field text (length-prefixed, often leading CRLF)
0x4023	script	OpenScript bytecode
0x4034	vertices	Point array route
0x4050	point-array setter	Runtime only; stores count + points
0x4015	idNumber	Object ID

Polygon Geometry (solved)

Draw records are 14 00 00 subtype-0 chunks with +0x0c = 04 00. Point count is the word at +0x1a; x,y twip pairs start at +0x22. Fill color dword is at +4 (e.g. 0xC0C0C0 silver). Screen coordinates = (stored * 96 + 720) / 1440 (or stored/15 when DPI factors are 96). Confirmed on P430 pin tips and chip body. Individual pin positions use an index at +0x52.

Arc Geometry

Type 0x11. Four twip pairs at +0x25/+0x27, +0x29/+0x2B, +0x2D/+0x2F, +0x31/+0x33. Bounding box at +0x0F. Fill color at +0x18. Painter is TB80BAS segment 0x7B. ToolBook does not call GDI!PIE; pie wedges are computed via GDIELLIPSEPOINTFROMANGLE then filled.

Object Type Table

TB80UTL.DLL at file 0xDED6 contains the classification table: ComboBox, Stage, Picture, Hotword, RecordField, PaintObject, Curve, AngledLine, Pie, Arc, IrregularPolygon, Polygon, Line, RoundedRectangle, Ellipse, Group, Field, Button, Rectangle, then Page, Background, Book. Resource kinds after *MEMBLOCK* include cel, anim, ref, sharedscript, clip, font, menubar, palette, cursor, icon, bitmap.

Open Items

- Absolute bounds are not stored; position is relative to a parent.
- Many connecting lines and detail strokes are computed from object bounds by segment 0x5B rather than

stored.
• Curve type ID is 0x05C7; control structure for embedded point groups still incomplete.
• Record-to-bounds extractor (the code that reads a 14 00 00 chunk and writes object+0xF) has not been located.

OpenScript Decoder

Bytecode expansion and extracted handlers

Decoder Status

Keyword offset table is at byte 2 + count*4 in resource 0x77 (TB80CMP). Back-references 0xC0–0xFF copy from the compressed input stream (not the output). Decoder openscript_decode.py produces readable handlers.

Example Recovered Handlers (9e954d01.tbk)

• if page = false then go to page …
• send next
• go to page "P400"
• go to page "P310" of book "nida81.tbk"

All 76 handlers from Logic Families extract correctly. System books (nc81*) have no "to handle" OpenScript; nida81.tbk has ~100 buttonClick handlers in the same format.

Notes

Handlers are stored inside page records. The modern readers (Maybe2/4, Brickwall, Foolbook) already prefer decoded "go to page" / "send" / nextpage for navigation and quizzes. Full OpenScript execution (conditionals, scoring, system calls) is not yet implemented.

SoftICE Dynamic Notes

Live object layouts and paint traces

Window Classes (TB80RTM)

TBKMain, TBKClient, TBKChildWindow, TBKReserved_1, SchedulerWndTB80. A lesson click is delivered to TBKClient, not TBKChildWindow. BMSG does not see the 16-bit queue; BPX on the TBKClient proc does.

Polygon Painter Confirmation

TB80BAS segment 0x82 paints a polygon. GDI!POLYGON count is at object+0x30 (word), point array far pointer at object+0x32 (4 bytes each, stored twips). PaintPolygon = FUN_1408_0376. ScaleTwipsToScreen = FUN_1008_01dd (seg 2 offset 0x1DD). With DS:01A6/01A8 = 96, screen = stored/15. Breakpoint fires per polygon; leave it set while scrolling freezes SoftICE.

Arc and Rectangle

Arc painter: TB80BAS segment 0x7B, CALL GDI!ARC. Object far pointer at [BP+04]. Bounding box stored as twips at object+0x0F. Fill color at +0x18. Rectangle painter (seg 0x83) reads bounds at +0x0F and calls GDI!RECTANGLE. Chip bodies are the small rectangles.

Dump Caveats

Early /DUMP captures were SoftICE console logs, not target memory. With expert mode and careful addressing, real page and object dumps were obtained (ASYM_TpID, Page#, property blocks, live point arrays).

Clean-Room Runtimes

What already works and what remains

Existing Readers

• **viewer / viewer2 / viewer3**: HTML frame + XML/JSON books; page marker walk, text, quoted jumps.
• **Maybe2 / Maybe4**: lessonshell JSON with Next/Back, quiz choices, decoded script; path manager and quiz helpers.
• **runtime/ and runtime_cpp/**: Python and C++ packages that parse a book and emit HTML or drive a simple player.
• **Brickwall / Foolbook**: self-contained GUI (tbk_gui.py / python -m foolbook --gui) that serves raw .tbk or JSON shell.

What Works Today

Textual content, navigation, and quizzes for books that resemble Logic Families. System-book stubs supply path manager, bookmarking, and popup behavior. Polygon renderer for kind=0400 records (P430 chip example).

Still Open

• Full vector painting for arcs, curves, computed brackets, and placed bitmaps.
• OpenScript execution and scoring.
• Cross-book references and system-book integration beyond stubs.
• NE host that fully opens a book under emulation.

TB80BAS Known Names

Objects, properties, painters, twip scale

Confirmed Renames

Original	Name
FUN_1408_0376	PaintPolygon
FUN_13e8_00e6	PaintPolyline
FUN_10a0_0558	PaintArc
FUN_10a0_0458	PaintArcAlt
FUN_1020_03e4	PaintRectangle
FUN_1008_01dd	ScaleTwipsToScreen
FUN_12d0_03ec	SetProperty
FUN_1408_00d8	SetPointArray
FUN_12d0_086c	SetupPaintRect
FUN_1008_03f6	SetScriptProperty
FUN_1070_1774	ValidateProperty
FUN_1070_1b3c	CheckVertices
FUN_1230_2f1c	PaintEllipse
FUN_13d8_0066	PaintEllipseAlt
FUN_1230_2278	PaintRoundRect
FUN_1230_26f4	PaintRoundRectAlt
FUN_13e0_015a	PaintRoundRectAlt2
FUN_1230_2b5a	PaintArc2
FUN_10e0_208e	SetMultiProperty
FUN_10e8_33e8	SetMultiProperty2

Nameable Functions (pattern)

Full list is in bas_nameable.tsv (copied into this directory). Below is a sample of the first entries.

category	function	confirmed_name
GDI	FUN_1008_01dd	ScaleTwipsToScreen
GDI	FUN_1008_0290	—
GDI	FUN_1008_03f6	—
GDI	FUN_1008_0ac8	—
GDI	FUN_1010_3138	—
GDI	FUN_1010_43f2	—
GDI	FUN_1018_0000	—
GDI	FUN_1018_0cbe	—
GDI	FUN_1018_0e60	—

category	function	confirmed_name
GDI	FUN_1020_0176	—
GDI	FUN_1020_0b96	—
GDI	FUN_1020_16de	—
GDI	FUN_1028_11e2	—
GDI	FUN_1028_23ec	—
GDI	FUN_1028_37ae	—
GDI	FUN_1030_08ec	—
GDI	FUN_1030_0b80	—
GDI	FUN_1038_08a0	—
GDI	FUN_1038_08fc	—
GDI	FUN_1060_143e	—
GDI	FUN_1098_0514	—
GDI	FUN_1098_0818	—
GDI	FUN_1098_0b88	—
GDI	FUN_1098_0be0	—
GDI	FUN_1098_0c46	—
GDI	FUN_1098_0cda	—
GDI	FUN_1098_0d86	—
GDI	FUN_1098_0daa	—
GDI	FUN_1098_0dce	—
GDI	FUN_1098_0ec0	—
GDI	FUN_1098_0ee2	—
GDI	FUN_10a0_0018	—
GDI	FUN_10a0_00e6	—
GDI	FUN_10a0_0178	—
GDI	FUN_10a0_038a	—
GDI	FUN_10a0_03fa	—
GDI	FUN_10a0_0458	PaintArcAlt
GDI	FUN_10a0_04da	—
GDI	FUN_10a0_0558	PaintArc
GDI	FUN_10a0_061e	—

(showing first 40 of many; see TSV for complete list)

Notes

Rename script: rename_bas.py. Run from Ghidra Script Manager on the corresponding project. Confirmed names are the ones that have been manually verified against GDI calls, SoftICE traces, or clear property-ID patterns.

TB80BMP Known Names

Bitmaps, DIB, palette, LZ77 expand

Confirmed Renames

Original	Name
FUN_1078_05c2	CreateDibBitmap
FUN_1090_0558	CreateDibBitmapAlt
FUN_1098_0874	CreateDibBitmapAlt2
FUN_10a0_16ee	CreateDibBitmapAlt3
FUN_1048_0116	BitBltBmp
FUN_1048_1250	BitBltBmp2
FUN_1048_2c5e	BitBltBmp3
FUN_1080_0194	BitBltBmp4
FUN_1010_18a4	SelectPaletteBmp
FUN_1030_03e4	GetDcBmp
FUN_1080_0548	SelectPaletteBmp2
FUN_1000_0031	Lz77Expand
FUN_1000_0162	Lz77ExpandCaller
FUN_1000_0752	Lz77ExpandCaller2

Nameable Functions (pattern)

Full list is in bmp_nameable.tsv (copied into this directory). Below is a sample of the first entries.

category	function
File	FUN_1008_0432
File	FUN_1090_0512
File	FUN_1090_0558
File	FUN_10a0_0eee
File	FUN_10a0_0f38
File	FUN_10a0_16ee
File	FUN_10a0_194e
File	FUN_10a0_19ae
GDI	FUN_1008_0500
GDI	FUN_1008_07a8
GDI	FUN_1008_0898
GDI	FUN_1010_1e72
GDI	FUN_1010_1eb0
GDI	FUN_1010_21ce
GDI	FUN_1048_0116
GDI	FUN_1048_1090

category	function
GDI	FUN_1048_1132
GDI	FUN_1048_26de
GDI	FUN_1048_28a8
GDI	FUN_1048_2940
GDI	FUN_1068_005e
GDI	FUN_1068_00d2
GDI	FUN_1070_04f2
GDI	FUN_1070_0a48
GDI	FUN_1078_0a0a
GDI	FUN_1080_011c
GDI	FUN_1080_02c4
GDI	FUN_1080_0548
GDI	FUN_1080_057a
GDI	FUN_1080_0722
GDI	FUN_1080_094e
GDI	FUN_1088_05ee
GDI	FUN_1088_06d4
GDI	FUN_10b8_002c
Memory	FUN_1008_07a8
Memory	FUN_1008_0c20
Memory	FUN_1008_17d0
Memory	FUN_1008_1a70
Memory	FUN_1030_06ee
Memory	FUN_1038_011e

(showing first 40 of many; see TSV for complete list)

Notes

Rename script: rename_bmp.py. Run from Ghidra Script Manager on the corresponding project. Confirmed names are the ones that have been manually verified against GDI calls, SoftICE traces, or clear property-ID patterns.

TB80CMP Known Names

OpenScript compiler resources and expand

Confirmed Renames

Original	Name
FUN_1010_0ee6	LoadCompilerResource
FUN_1010_1390	LockCompilerResource
FUN_1038_0dea	LockCompilerResource2
FUN_1040_0970	ExpandTextCore
FUN_1010_0070	CompilerStrLen
FUN_1010_1774	CompilerStrCpy
FUN_1010_1a28	CompilerStrCat
FUN_1038_06f8	CompilerStrHelper

Nameable Functions (pattern)

Full list is in cmp_nameable.tsv (copied into this directory). Below is a sample of the first entries.

category	function
Memory	FUN_1008_1eea
Memory	FUN_1008_1f6c
Memory	FUN_1010_1030
Memory	FUN_1010_1352
Memory	FUN_1010_1898
Memory	FUN_1010_1b56
Memory	FUN_1040_0000
Memory	FUN_1040_0e1c
Memory	FUN_1040_0f08
Memory	FUN_1040_0fa6
Memory	FUN_1078_0400
Memory	FUN_1078_0530
Memory	FUN_1080_12ba
Memory	FUN_1088_15ba
Memory	FUN_10a8_004e
Memory	FUN_10a8_0130
Memory	FUN_10f8_0000
Message	FUN_1088_0fca
Property	FUN_1008_1848
Property	FUN_1010_0fda
Property	FUN_1028_0000
Property	FUN_1028_8ce8

category	function
Property	FUN_1038_0b24
Property	FUN_1048_18a8
Property	FUN_1070_0c60
Property	FUN_1078_0530
Property	FUN_1080_0b4c
Property	FUN_1088_15ba
Property	FUN_1088_1984
Property	FUN_10a8_12c4
Property	FUN_10a8_199a
Property	FUN_10c0_0c24
Property	FUN_10c0_0cb0
Property	FUN_10e0_0000
Property	FUN_10e0_00d4
Resource	FUN_1010_0f2a
Resource	FUN_1040_0f08
String	FUN_1008_2000
String	FUN_1010_0ee6
String	FUN_1010_0f2a

(showing first 40 of many; see TSV for complete list)

Notes

Rename script: rename_cmp.py. Run from Ghidra Script Manager on the corresponding project. Confirmed names are the ones that have been manually verified against GDI calls, SoftICE traces, or clear property-ID patterns.

TB80RTM Known Names

Main engine, windowing, page paint

Confirmed Renames

Original	Name
FUN_10e0_0050	PaintRectangle
FUN_1398_05c6	PaintRectangleAlt
FUN_1070_0b32	BitBltHelper
FUN_10d0_2d4e	BitBltHelper2
FUN_10d0_2fa0	BitBltHelper3
FUN_10e0_0dea	BitBltHelper4
FUN_1130_32ce	BitBltHelper5
FUN_1010_1a0a	SendMessageHelper

Nameable Functions (pattern)

Full list is in rtm_nameable.tsv (copied into this directory). Below is a sample of the first entries.

category	function
File	FUN_1008_485e
File	FUN_1090_048c
File	FUN_1150_0000
File	FUN_1190_11e8
File	FUN_1190_154c
File	FUN_1240_0888
File	FUN_1240_1232
File	FUN_1240_12b4
File	FUN_1240_13c2
File	FUN_1268_0882
GDI	FUN_1008_068c
GDI	FUN_1040_0338
GDI	FUN_1040_041a
GDI	FUN_1058_08bc
GDI	FUN_1058_098c
GDI	FUN_1068_0000
GDI	FUN_1068_0034
GDI	FUN_1070_0b32
GDI	FUN_1070_14d4
GDI	FUN_1090_0284
GDI	FUN_10b8_1588
GDI	FUN_10b8_17dc

category	function
GDI	FUN_10b8_26f0
GDI	FUN_10b8_3192
GDI	FUN_10c8_125a
GDI	FUN_10d0_0fd6
GDI	FUN_10d0_1d4a
GDI	FUN_10d0_2d4e
GDI	FUN_10d0_2fa0
GDI	FUN_10e0_0050
GDI	FUN_10e0_022a
GDI	FUN_10e0_0d5c
GDI	FUN_10e0_0dea
GDI	FUN_1130_2404
GDI	FUN_1130_32ce
GDI	FUN_1130_35b0
GDI	FUN_1130_366e
GDI	FUN_1130_38e8
GDI	FUN_1130_3934
GDI	FUN_1138_060e

(showing first 40 of many; see TSV for complete list)

Notes

Rename script: rename_rtm.py. Run from Ghidra Script Manager on the corresponding project. Confirmed names are the ones that have been manually verified against GDI calls, SoftICE traces, or clear property-ID patterns.

TB80UTL Known Names

Object type table and helpers

Confirmed Renames

Original	Name
FUN_1000_093b	UtlAlloc
FUN_1018_1472	UtlLock
FUN_10b8_0eac	UtlAlloc2
FUN_10c0_028c	UtlLock2
FUN_1008_0504	UtlStrCpy
FUN_10b8_0000	UtlStrLen
FUN_10c8_001e	UtlStrCat

Nameable Functions (pattern)

Full list is in utl_nameable.tsv (copied into this directory). Below is a sample of the first entries.

category	function
File	FUN_1030_0290
File	FUN_1030_03a4
File	FUN_1030_03f0
GDI	FUN_1060_01d6
Memory	FUN_1000_093b
Memory	FUN_1008_0504
Memory	FUN_1018_00c9
Memory	FUN_1018_0230
Memory	FUN_1018_06e0
Memory	FUN_1018_06f2
Memory	FUN_1018_074a
Memory	FUN_1018_1472
Memory	FUN_1018_1682
Memory	FUN_1018_24fc
Memory	FUN_1030_0290
Memory	FUN_1030_03f0
Memory	FUN_1038_0d32
Memory	FUN_1038_0d90
Memory	FUN_1038_0df8
Memory	FUN_1038_0f5c
Memory	FUN_1078_0000
Memory	FUN_1078_0c8c
Memory	FUN_1080_0122

category	function
Memory	FUN_1080_018e
Memory	FUN_1080_04fe
Memory	FUN_10a8_0520
Memory	FUN_10b8_0e90
Memory	FUN_10c0_030a
Message	FUN_1018_06e0
Message	FUN_1078_01b2
Message	FUN_1078_0274
Property	FUN_1018_06e0
Property	FUN_1018_126d
Property	FUN_1018_284e
Property	FUN_1088_04e2
Property	FUN_10c8_0000
Resource	FUN_1030_03a4
Resource	FUN_1038_0800
String	FUN_1000_093b
String	FUN_1008_0504

(showing first 40 of many; see TSV for complete list)

Notes

Rename script: rename_utl.py. Run from Ghidra Script Manager on the corresponding project. Confirmed names are the ones that have been manually verified against GDI calls, SoftICE traces, or clear property-ID patterns.

TB80W16 Known Names

Thunk layer and graphic IDs

Confirmed Renames

Original	Name
FUN_1010_1914	W16Alloc
FUN_1058_0f14	W16Lock
FUN_1058_2568	W16Alloc2
FUN_10b0_0060	W16Lock2
FUN_1030_003a	W16StrCpy
FUN_10b0_022c	W16StrLen
FUN_10c0_013a	W16StrCat
FUN_10c0_0310	W16StrHelper

Nameable Functions (pattern)

Full list is in w16_nameable.tsv (copied into this directory). Below is a sample of the first entries.

category	function
File	FUN_10b0_0144
File	FUN_10b0_2d90
File	FUN_10e8_01c0
GDI	FUN_1010_034c
GDI	FUN_10b0_0144
GDI	FUN_10e8_01c0
Memory	FUN_1010_034c
Memory	FUN_1010_035c
Memory	FUN_1010_1914
Memory	FUN_1010_1cde
Memory	FUN_1058_0de0
Memory	FUN_10a0_0ee2
Memory	FUN_10b0_008a
Memory	FUN_10b0_0144
Memory	FUN_10c0_0016
Memory	FUN_10c0_00d0
Memory	FUN_10f0_01ba
Message	FUN_1038_0992
Message	FUN_1058_0b2c
Message	FUN_1058_0de0
Message	FUN_10a0_0ee2
Property	FUN_1010_054e

category	function
Property	FUN_1038_04ee
Property	FUN_1070_0000
Property	FUN_10b0_0144
Property	FUN_10e8_0000
Property	FUN_10e8_01c0
Resource	FUN_1068_0544
String	FUN_1010_034c
String	FUN_1010_035c
String	FUN_1010_0632
String	FUN_1010_08c0
String	FUN_1010_10ad
String	FUN_1010_1154
String	FUN_1038_000c
String	FUN_1038_0992
String	FUN_1058_0de0
String	FUN_1068_0894
String	FUN_10a0_077c
String	FUN_10a0_0ee2

(showing first 40 of many; see TSV for complete list)

Notes

Rename script: rename_w16.py. Run from Ghidra Script Manager on the corresponding project. Confirmed names are the ones that have been manually verified against GDI calls, SoftICE traces, or clear property-ID patterns.

TB80LNL Known Names

LNL helpers

Confirmed Renames

Original	Name
FUN_1018_04a6	LnlAlloc
FUN_1050_043a	LnlLock
FUN_1070_203e	LnlAlloc2
FUN_10d8_097e	LnlLock2
FUN_1040_344c	LnlStrCpy
FUN_1040_40f0	LnlStrLen
FUN_1090_0130	LnlStrCat

Nameable Functions (pattern)

Full list is in lnl_nameable.tsv (copied into this directory). Below is a sample of the first entries.

category	function
File	FUN_1010_0094
File	FUN_1010_0408
File	FUN_1020_0084
File	FUN_1050_04c2
File	FUN_1090_092a
File	FUN_10d8_05fe
File	FUN_1100_043a
GDI	FUN_10a0_35d8
Memory	FUN_1010_0094
Memory	FUN_1018_051c
Memory	FUN_1018_05c0
Memory	FUN_1020_0744
Memory	FUN_1028_0440
Memory	FUN_1038_05e4
Memory	FUN_1048_0000
Memory	FUN_1050_0002
Memory	FUN_1050_04c2
Memory	FUN_1070_010e
Memory	FUN_1070_203e
Memory	FUN_1070_20c0
Memory	FUN_10d8_0000
Memory	FUN_10f8_170c
Memory	FUN_10f8_18c8

category	function
Memory	FUN_10f8_1cd2
Memory	FUN_1110_0000
Memory	FUN_1110_00cc
Message	FUN_1018_0096
Message	FUN_1020_012a
Message	FUN_1028_0050
Message	FUN_1028_00d0
Message	FUN_1030_0238
Message	FUN_1030_0ede
Message	FUN_1030_0f92
Message	FUN_1030_0fbe
Message	FUN_1030_10dc
Message	FUN_1040_00d4
Message	FUN_10a0_35d8
Message	FUN_10b0_0760
Message	FUN_10c8_396e
Message	FUN_10d8_19d2

(showing first 40 of many; see TSV for complete list)

Notes

Rename script: rename_lnl.py. Run from Ghidra Script Manager on the corresponding project. Confirmed names are the ones that have been manually verified against GDI calls, SoftICE traces, or clear property-ID patterns.

TB80MM Known Names

Multimedia notifications

Confirmed Renames

Original	Name
FUN_1070_03f4	MmSendMessage
FUN_1088_0200	MmSendMessage2
FUN_1088_0642	MmSendMessage3
FUN_1088_09f0	MmCreateWindow
FUN_1048_00d0	MmAlloc
FUN_1098_006a	MmAllocStr
FUN_1098_0134	MmStrCpy

Nameable Functions (pattern)

No nameable TSV available for this module.

Notes

Rename script: rename_mm.py. Run from Ghidra Script Manager on the corresponding project. Confirmed names are the ones that have been manually verified against GDI calls, SoftICE traces, or clear property-ID patterns.

TB80RCR Known Names

RCR graphics

Confirmed Renames

Original	Name
FUN_1008_158c	RcrBitBlt
FUN_1008_164c	RcrBitBlt2
FUN_1008_170e	RcrBitBlt3
FUN_1008_077a	RcrSendMessage
FUN_1008_0e94	RcrSendMessage2
FUN_1008_11aa	RcrSendMessage3
FUN_1008_08e2	RcrAlloc
FUN_1008_2190	RcrAlloc2
FUN_1008_aa62	RcrStrCpy

Nameable Functions (pattern)

No nameable TSV available for this module.

Notes

Rename script: rename_rcr.py. Run from Ghidra Script Manager on the corresponding project. Confirmed names are the ones that have been manually verified against GDI calls, SoftICE traces, or clear property-ID patterns.

TB80FLT Known Names

FLT graphics

Confirmed Renames

Original	Name
FUN_1028_14be	FltBitBlt
FUN_1040_071a	FltBitBlt2
FUN_1028_0390	FltSendMessage
FUN_1028_2316	FltSendMessage2
FUN_1028_26ac	FltSendMessage3
FUN_1010_0e98	FltAlloc
FUN_1010_20b6	FltAlloc2
FUN_1000_093b	FltStrCpy

Nameable Functions (pattern)

No nameable TSV available for this module.

Notes

Rename script: rename_flt.py. Run from Ghidra Script Manager on the corresponding project. Confirmed names are the ones that have been manually verified against GDI calls, SoftICE traces, or clear property-ID patterns.

TB80NET Known Names

Network layer

Confirmed Renames

Original	Name
FUN_1038_01b4	NetSendMessage
FUN_1038_0484	NetSendMessage2
FUN_1040_0000	NetSendMessage3
FUN_1008_17c8	NetCreateWindow
FUN_1008_0862	NetAlloc
FUN_1008_136c	NetAlloc2
FUN_1020_0410	NetStrCpy
FUN_1020_0a10	NetStrCpy2

Nameable Functions (pattern)

No nameable TSV available for this module.

Notes

Rename script: rename_net.py. Run from Ghidra Script Manager on the corresponding project. Confirmed names are the ones that have been manually verified against GDI calls, SoftICE traces, or clear property-ID patterns.

Ledger

Stable print: 1.0.0r1. Edits after that tag are experimental until the next release. Written 2026-10-13. The engine notes cite TBpipeline at `a106ef0` unless a claim says otherwise.

Id | Grade | Statement

WV-001 | Known | These lesson files are ToolBook II 8.1 shells, book version `01 00 00 08`.

WV-002 | Known | The book stream starts after a `0xC00` PE stub, at `ZZZTBKBOOKLOC` and magic `0x22222222`.

WV-003 | Known | Page records begin `80 65 00 05 00 02 02`. Widgets inside them begin `14 00 00`.

WV-004 | Known | Polygon kind `04 00` is decoded, and P430 draws from it.

WV-005 | Known | The clean-room reader is not the engine.

WV-006 | Known | The engine host has not opened a lesson book.

WV-007 | Thought | `CDBFILEOPEN` expands the disk record and writes the live object.

WV-008 | Thought | A real Win16 return is the only way the Unicorn host gets further.

WV-009 | Conjecture | ToolBook 11.5 can export an 8.1 lesson as `.TBKX`.

WV-010 | Not true | On-disk `14 00 00 01` becomes `14 00 00 11` in memory.

WV-027 | Known | ToolBook 8.1 calls 16-bit `USER`, `GDI`, and `KERNEL`, not `USER32`.

WV-028 | Known | Windows 98 can step this engine. 32-bit XP can run it. 64-bit Windows cannot.

WV-029 | Known | Published users are training shops and the Navy lessons on disk, not a living authoring market.

WV-030 | Thought | Someone outside this set who still holds a native 8.x book has the same problem.

Known

WV-001

The courses in this work are Nida Corporation electronics lessons, written as Click2learn ToolBook II 8.1 shells. The book version word is `01 00 00 08`. This Nida is not the drug-abuse institute that shares the letters.

Evidence: the shell header, the version word, and the lesson text. Logic Families is `9e954d01.tbk`. Falsify: a lesson in the set whose book stream is a different version, or whose text is not the electronics course.

WV-002

A lesson file begins with a `0xC00` PE stub that has no useful code. The book stream is marked by `ZZZTBKBOOKLOC` and magic `0x22222222`, then `03 4A 42 4F`. The opener is `tbload.exe` through `MINSTB32.dll`.

Evidence: the same layout on the lesson files that have been opened, including `9e954d01.tbk` at file offset `0x9ec`. Falsify: a lesson in the set whose JBO stream is not behind that marker.

WV-003

The only page records found start with `80 65 00 05 00 02 02`. Inside a page, widget chunks start with `14 00 00`. A common property header is `14 00 00 01`. Field text is a length-prefixed `Text` property. Property ids recovered from `TB80BAS` resources include bounds `0x4004`, text `0x402e`, script `0x4023`, and vertices `0x4034`.

Evidence: the marker search, and NE resource type `0x2d1` / id `0x2d0` in `TB80BAS`. Falsify: a page that displays in the real runtime and does not carry that marker, or a property id in that resource that does not match the name assigned here.

WV-004

Polygon records use kind `04 00`. The count and the twip pairs are at fixed offsets in that record. Screen position is `(stored * 96 + 720) / 1440`, which is twips / 15 at 96 dpi. P430, the chip, draws from this, including the pins. In the

live object, bounds are at `+0x0F`, the point count at `+0x30`, and the point array at `+0x32`. The painter is in `TB80BAS` segment `0x82`.

Curves on P420 can be drawn as separate files. They are not placed on the page. Neither are the polygons.

Evidence: the record layout, a live `GDI Polygon` trace, and the reader output for P430 and P420. Falsify: a P430 point that the real runtime paints and this layout misses, or a live object whose count is not at `+0x30`.

WV-005

The clean-room reader walks `80 65` pages, pulls field text, and follows quoted page ids such as `P400`. On `9e954d01.tbk` that is 129 pages. It also extracts 66 Windows BMPs from the book and one tied to P1290, and it draws 18 pages of shapes (23 files, because a page can have more than one curve). The pictures and the shapes are not placed. Navigation is the quoted id, not the bytecode. Quizzes are scraped text, not scored.

Evidence: a TBpipeline `1.0.0r4` run on that book. The debug folder's `render.html` is this reader. Stage 2a runs because the engine did not open the book. Falsify: a handler whose target is only a bytecode operand and is still followed correctly, or a page whose bitmap is placed by this reader.

WV-006

`TB80RTM` is entered at segment 2 offset `0x35A`. As of TBpipeline `a106ef0`, the experimental host gets past the old `0x20:0x36c` fault, opens `TB80.INI` and `TB80enu`, binds 580 ToolBook DLL entries, and then exits through DOS with code 0. The book is not among the files it opened. `USER.266` is `SetMessageQueue`. Reaching it is not startup finishing.

An earlier integrated run, before that commit, died 18 bytes after the entry. The instruction was `les ax, [di-66h]`. The address was unmapped. No Win16 call had run.

Evidence: `nehost/rtm - pass 06/PASS06.txt` at `a106ef0`, and the `1.0.0r4` launch log from the earlier run. Falsify: a host log from this tree whose open list contains the lesson `.tbk` and whose fault is not an exit.

WV-011

These ordinals are fixed in the unpacked images. `TB80BAS`: `CDBINIT` 1, `CDBFILEOPEN` 8, `CDBFILESAVE` 9, `CDBVIEWPAGE` 15, `CDBVIEWBKGND` 18. `TB80CMP`: `COMPILERCOMPILETEXT` 1, `COMPILEREXPANDTEXT` 3, `CFUNEXECUTE` 8, `CFUNEVALUATE` 9. `TB80JPG` still carries the version-resource name `TB60JPG.DLL`. `TB80LNL` is the Lionel media engine. `TB80RTM` window classes include `EGYPT*`, `TBKMain`, `TBKClient`, and `SchedulerWndTB80`.

Evidence: export tables, version resources, and window-class strings. Falsify: an unpacked 8.1 image in which one of these ordinals is a different export.

WV-012

The call at `TB80RTM` segment 22 offset `0x8AA` is `TB80UTL` ordinal 3, `GETSTRMGRSTRING`, five Pascal words. It is not `KERNEL.3` `GetVersion`. String id `0x0FBD` is the product-name id the startup checks. An empty buffer at that check is one path into the C runtime exit.

The words `"ToolBook II"` written by the host are a non-empty stub. They were not read out of the DLL.

Evidence: the reloc at that site, and the host note in TBpipeline `a106ef0`. Falsify: a disassembly of that site that is `KERNEL.3`, or a capture of id `0x0FBD` whose text is the stub's sentence and was taken from the binary.

WV-013

`.TBKX`, from about ToolBook 9.5, is a ZIP of the media plus an XML description of pages, objects, bounds, layers, and properties. JohnDoe1 described a hidden command in ToolBook 11.5 that writes it. 11.5 is 32-bit, needs MSXML 4, and the trial cannot be activated. A book saved by the trial forever shows the trial banner in the runtime. Versions before 9.0 are 16-bit and do not run on 64-bit Windows. The steps for 8.1 to 11.5 had not arrived on 2026-10-12, and no lesson has been opened in 11.5 here.

Evidence: JohnDoe1's description of the format and the trial, recorded in `SME/README.md`. The personal name and the site are not in this repository. Falsify: a 11.5 that writes `.TBKX` as something other than ZIP plus that XML, or a trial save that

does not set the banner.

WV-014

The 1.5 `PHYSSIZE` index is not present in these 8.1 books. `TB80RMT` is not on the path that opens a lesson. The NE images are SLR Optloader wrappers. OpenScript keyword tables are `TB80CMP` resources `0x77` and `0x78`. A decoder can turn some handlers into text, including `go to page` and `send next`. It is not an interpreter.

Evidence: the search of the 8.1 books, the opener's import set, the unpacker, and the decoder notes. Falsify: an 8.1 lesson that contains a `PHYSSIZE` index, or a lesson open that loads `TB80RMT` before the first page.

WV-027

ToolBook II 8.1 does its work as Win16. The 32-bit launcher hands the book to `TB80RTM`. Painting goes through 16-bit `GDI`. The startup list of missing `USER32`, `GDI32`, and `KERNEL32` is not a broken install. Those are not the DLLs this runtime calls.

Evidence: the NE images, the ordinal imports, and the pipeline startup log that reports those 32-bit names absent and continues. The longer account is `environments.md`. Falsify: a lesson open on Windows 98 or XP that calls `USER32` for `SetMessageQueue` or `Polygon`.

WV-028

Windows 98 is the decompile floor because SoftICE can step the 16-bit segments there. 32-bit Windows XP runs the same lessons inside `NTVDM`, and it is a poor place to single-step `TB80RTM` from the 32-bit launcher. 64-bit Windows has no `NTVDM` and cannot run 8.1 at all. ToolBook 9.0 replaced this engine with a 32-bit one. 11.5 is that later program.

Evidence: the project's own choice of machines, Microsoft's description of `NTVDM`, and the version history (9.0 in 2007, 11.5 in December 2012, support ended 31 December 2021). Falsify: a 64-bit system that starts `TB80RTM` without emulating a 16-bit CPU, or a single-step of this engine on XP that sees object `+0x0F` as clearly as Windows 98.

WV-029

The books in this project were written by Nida Corporation for Navy electronics training. Outside that set, published accounts name other training users of the authoring tools: Northrop Grumman on Air Force aircrew lessons, a 1997 Navy demonstration aboard the USS Princeton, a Pensacola course-designer pilot, an Air Force course-management pilot at Randolph, Buckman Laboratories' learning center, and a UK user conference in 1994. A marketing line that lists AT&T;, Lucent, MetLife, and Boeing is not a customer list. There is no present market for new ToolBook courses. Support ended in 2021.

Evidence: the lesson copyright pages, and the sources in `audience.md`. Falsify: a current vendor offering ToolBook 8.1 authoring, or one of those marketing names documented as an 8.1 runtime site.

Thought

WV-007

Disk records are the compact form. `CDBFILEOPEN` (`TB80BAS` ordinal 8) is what expands them. Bounds end up at object `+0x0F`. The six words after `14 00 00 01` are not those bounds. The instruction that performs the write has not been found, which is why placement still fails.

Evidence for the shape of the claim: live bounds at `+0x0F`, and no bounds in the six words. The expansion itself has not been traced. Falsify: a trace of ordinal 8 that never writes `+0x0F`, or those six words turning out to be the rectangle.

WV-015

A click does not call a function pointer stored in the book. The runtime sends a message and walks until a handler takes it: object, group, page, background, book, then the system books in list order, then `TB80RTM`. `forward` continues the walk. Reader and Author are a mode, not a second engine. `TB80EDT.DLL` is named by the runtime and is not in this set.

This is the published ToolBook model, and it matches the export names. It has not been single-stepped through a lesson click.

Falsify: a click that jumps to an address stored in the page record, or a lesson that runs with no system book and still draws the Nida chrome.

WV-016

The system books (`tb80r.sbk`, `tb80hyp.sbk`, `tb80anm.sbk`, and the four `nc81*.sbk` files) are ordinary books. They hold the shell: path, quiz, and status behavior that is not in `TB80RTM`. Their handlers have not been fully read. `linkDLL` modules (`TBDLG`, `TBWIN`, `TBDOS`, `TBREG`, `TBDB3`, `TBPDX`, `TBCBT`) are not loaded unless a handler asks.

Falsify: a system book whose stream is not a book, or a lesson open that loads `TBCBT` before any script runs.

WV-008

The Unicorn host is a CPU with a fake Win16. A stub that returns `AX=1` is not a successful call. Callers use that value as a handle, a pointer, or an `HRESULT`, and the next fault is the lie. Getting further means returning a real object for each imported ordinal, and entering `TB80RTM` only after the loader has set the data segment, the stack, and the startup parameters. ToolBook's own DLLs should stay real code. `KERNEL`, `USER`, and `GDI` have to be implemented for the ordinals the imports name.

The host is already that design, done one crash at a time. That is why it looks busy and does not open a book.

Falsify: a host that returns `AX=1` for the remaining ordinals and still opens `9e954d01.tbk`.

WV-017

Later live `80 65` objects were seen with `14 00 00 02`. The working idea is that the last byte is a flag, and `02` is a live value that is not the disk value `01`. What the flag means is not known.

Evidence: the live dumps that showed `02`. No dump has shown the old `11`. Falsify: a live object whose header is `01` after `CDBFILEOPEN`, or a meaning for `02` that the painter ignores.

WV-030

The practical audience now is whoever still has a native 8.x book and cannot open it on 64-bit Windows. That includes other CBT discs from the same era, not only these lessons. It does not include people choosing an authoring tool.

Evidence: WV-028, and the absence of a product. This remains a thought until a second corpus is actually in hand. Falsify: a native 8.x book that opens on 64-bit Windows without a 16-bit emulator, or a second corpus that does not use the same shell.

Conjecture

WV-009

If an 8.1 lesson opens in ToolBook 11.5, the hidden save will write a `.TBKX` whose XML is the same pages, objects, bounds, and scripts the `80 65` records hold. That would document the property block the 8.1 parser does not yet expand. It has not been tried. JohnDoe1 had not sent the steps as of 2026-10-12.

Falsify: 11.5 refusing the lesson, or a `.TBKX` that has the media and does not have the object tree.

WV-018

The DOS exit 0 in the `a106ef0` log is the C runtime leaving because some later init check failed. It is not a successful run that had nothing left to do. The product-name stub removed one empty-string exit. It did not finish startup.

Falsify: a log of that same path that paints a window or calls `CDBFILEOPEN` on the lesson before it exits.

WV-019

Picture widgets can be tied to the carved BMPs from the disk records alone, once WV-007 is known. Until then, carving a BMP does not place it.

Falsify: a placement found without the expanded property block, or an expanded block that still has no picture id.

Not true

These stay here so they are not discovered again.

WV-010

On-disk `14 00 00 01` becomes `14 00 00 11` in memory.

Believed because an early note compared a disk record with a SoftICE view of object 0. Not confirmed. Later live objects used `14 00 00 02`. Do not treat `11` as the expanded form. Retired 2026-10-12.

WV-020

`AX=1` from an unknown Win16 ordinal means the call worked.

It means the stub had nothing else to return. `CreateDispTypeInfo` failed because `1` is not a pointer. A pointer of `0000:0001` was called as code. Retired 2026-10-12.

WV-021

An unmapped read can be continued by mapping the page and filling it with `0xCB`.

That filler is what `les` then loads as a selector, and the selector is not in the descriptor table. The `int 0xd` at `0x20:0x36c` was this, not a new fact about ToolBook. Retired 2026-10-12.

WV-022

The files `bmp_bitmap.txt`, `dll_probe.txt`, and `scaffold.txt` inside a pipeline debug folder are a probe of the machine that just ran.

They are copies shipped in the package. One of them still names a path from the machine that first wrote it. Retired 2026-10-12.

WV-023

Stage 2 opened the book whenever `render.html` exists.

Stage 2a runs because stage 2 did not open the book. The reader is the fallback. The line `engine opens none` is the engine result. Retired 2026-10-12.

WV-024

The system books are only resource containers and hold no OpenScript.

They are books. Their scripts are unread, not absent. The claim that they contain no handlers is not supported. Retired 2026-10-13.

WV-025

Passing the entry fault means the book opened.

The `a106ef0` host passes the old fault and exits without opening the lesson. A longer call list is not an open. Retired 2026-10-13.

WV-026

The product-name string in the host was recovered from `TB80UTL`.

The host writes a non-empty placeholder so the empty-buffer check does not exit. The id `0x0FBD` is the claim. The letters in the placeholder are not. Retired 2026-10-13, at the same time the stub was written, so it would not be promoted later.

What ToolBook 8.1 thinks the machine is

A tangent on Win16, Windows 98, and Windows XP. Graded claims from it are WV-027 and WV-028. This file can wander. The ledger cannot.

The split personality

ToolBook II 8.1 is not one program. `TB80RUN` is a 32-bit process. It starts `TB80NET`, which loads `TB80RTM`, and the book is interpreted by 16-bit NE code. The 32-bit half exists to satisfy a 32-bit shell. The lesson runs in the 16-bit half.

That is why a startup inventory can list `USER32`, `GDI32`, and `KERNEL32` as absent and still be a normal run. Those are the 32-bit names. The engine calls `USER`, `GDI`, and `KERNEL` by 16-bit ordinal. `USER.266` is `SetMessageQueue` in that table. A Win32 `SetMessageQueue` is a different function in a different DLL, and 8.1 does not call it.

Win16, as the runtime sees it

Windows 3.x, and the Win16 layer that survived inside later Windows, gives a program three modules and a loader.

`KERNEL` owns the module list, the global heap, selectors, files, and resources. `USER` owns windows, classes, and the message queue. `GDI` owns pens, brushes, bitmaps, and the calls that actually paint. ToolBook's own DLLs sit beside those, not instead of them. `TB80BAS` can draw a polygon only by asking `GDI` to draw it. The live trace of P430 is a `GDI Polygon`, not a private framebuffer.

The loader's contract matters more than any single ordinal. Before the NE entry runs, a real Windows has already done `InitTask`: the data segment is `DS`, the stack is the one in the NE header, and the parameters are the instance, the previous instance, the command tail, and `nCmdShow`. The entry at segment 2 offset `0x35A` was written for that moment. Jumping there with empty registers is not starting ToolBook. It is starting the CPU at an address.

Inside that world, pointers are selector:offset. A local heap lives in the automatic data segment. A global allocation returns a handle that is not a pointer until it is locked. An ordinal that returns `1` has not returned a selector. The caller will load it and fault, and the fault will look like a new discovery about ToolBook. It is a discovery about the stub.

16-bit Windows is cooperatively scheduled, and on the systems that still have it, the Win16 programs share one box. One of them that does not pump messages stalls the others. ToolBook's message walk (object, group, page, background, book, system books) sits on top of that `USER` queue. It is not a second scheduler.

The files it looks for are also from that era. The host has seen it open `C:\TOOLBOOK\TB80.INI` and `C:\WINDOWS\asym\runtime\TB80.INI`, and the language file `TB80enu`. Those are Windows locations, not arguments the pipeline passed. On a real machine they are the install. In the host they are strings the code built because it believes it is running under Windows.

Windows 98

Windows 98 is still a DOS machine with the 16-bit user and GDI in the same world as the 32-bit thunks. A 16-bit segment is a segment the CPU is using, not a program being interpreted by `NTVDM`. SoftICE can set a breakpoint on `TB80BAS` and see the object at `+0x0F`. That is why this project treats Windows 98 as the decompile floor. It is not nostalgia. It is the last common system where the runtime and a 16-bit debugger occupy the same machine without a second kernel in between.

The 32-bit launcher still matters there. `TB80W32` and `TB80W16` are the two ends of a flat thunk. On Windows 98 that thunk is the system's own. The lesson does not become a 32-bit program because the launcher was one.

Windows XP

Windows XP is a different kernel. 16-bit code runs only inside `NTVDM`. Microsoft describes that process as one DOS environment, plus a Windows 3.x face: 16-bit `KERNEL`, `USER`, and `GDI`, and BIOS calls the subsystem answers. On a 32-bit XP the old instructions still execute on the CPU. `NTVDM` is not a full PC emulator. It is a virtual machine for the 16-bit environment.

From the book's point of view the ordinals did not change. `Polygon` is still `GDI`. `SetMessageQueue` is still `USER.266`. `TB80.INI` is still a file under the Windows directory. What changed is everything around the call. The message queue is the WOW queue, not the Windows 98 queue. A debugger attached to `TB80RUN` is attached to a 32-bit process. The instruction pointer it can single-step is not the one inside `TB80RTM`. That matches the working rule here: XP is the preferred machine for running a course, and it is a bad machine for stepping the engine. Windows 98 is the other way around.

64-bit Windows, including 64-bit XP, has no `NTVDM`. The long mode of x86-64 does not run 16-bit code beside 64-bit code. ToolBook 8.1 cannot start there. No compatibility tab fixes it. This is why a WinXP virtual machine, 32-bit, is the practical place to open these lessons, and why a Linux or Win11 host cannot host the engine by loading the DLLs.

ToolBook 9.0, in 2007, dropped that 16-bit engine and shipped as a 32-bit product. ToolBook 11.5 can run on 64-bit Windows because it is not this runtime. Opening an 8.1 book in 11.5 would be a different program reading an old file. It would not be `TB80RTM` on a newer desktop.

What not to conclude

A course that runs on XP has not proved that XP and Windows 98 present the same object at `+0x0F`. The painter was confirmed on Windows 98. The pipeline's Unicorn host is a third environment, and it is the least faithful of the three: it has the instruction set and a growing set of lies about `KERNEL`. A behavior seen only in the host is a behavior of the host until it is seen on Windows 98 or XP.

Who used ToolBook, and who would care now

Research notes, 2026-10-13. Names below are organizations in published sources. They are not people from this project's mail. Graded claims are WV-029 and WV-030.

What the product was

Asymetrix shipped ToolBook in 1990. Paul Allen founded the company after leaving Microsoft. The first versions were a Windows authoring tool in the same breath as HyperCard and, for a while, Visual Basic. The book metaphor is the product: a project is a book, a screen is a page, and OpenScript is the language.

The training edition arrived with ToolBook 3 in 1994. Question objects, scoring, and later an LMS (Librarian, then Ingenium) sat on top of the same book. Click2learn renamed the company. SumTotal Systems was the merger with Docent in 2004. The last ToolBook was 11.5, in December 2012. SumTotal ended sales and support on 31 December 2021.

Two products shared the name. Assistant was templates and no programming. Instructor was OpenScript. Native content was the Windows runtime these lessons use. DHTML content was the later web export. A remark about "ToolBook users" that does not say which of those it means is not about `TB80RTM`.

Who published sources actually name

These are contemporary accounts, not a census. A press release is a press release.

Known from the books in this project. Nida Corporation wrote the electronics lessons for Navy advanced technical training and shipped them as ToolBook II 8.1 shells. That use is not a clipping. The copyright pages and the lesson text are in the files.

Named in trade press as users or pilots of the authoring tools.

- Northrop Grumman, under an Air Force aircrew-training contract, adopted ToolBook II Assistant 6.5 to move faster-changing lessons off an older simulation system. An instructional designer there described video synchronized to highlighted text, aimed at manufacturing training. (FCW, November 1998.)
- The Navy. In 1997 Asymetrix demonstrated a ToolBook II lesson taken by a sailor on the USS Princeton, with results sent by satellite to Dahlgren, Virginia, and the company. The integrator named in the piece was Advanced Acoustics Concepts. The same season, Asymetrix had bought AimTech and allied with Raytheon E-Systems on Navy course modernization. (FCW, November 1997.)
- Naval Professional Development and Technology Center, Pensacola. ToolBook II was one of four authoring tools in a project for designers of Navy technical courses. (FCW, December 1996.)
- Air Education and Training Command, Randolph Air Force Base, planning a pilot of Asymetrix course management and a set of military-police modules. That article is about Librarian as much as about the authoring tool. (FCW, April 1998.)
- Buckman Laboratories, a specialty-chemical company, used ToolBook II Instructor and Ingenium for a learning center. The company's own case study claims 9,000 hours of multilingual training to 1,242 people in 1998. Treat the hour counts as the company's number.
- A 1994 UK ToolBook User Conference, and a Bristol medical-school paper on building lessons from templates without scripting. There was a user community outside the US military.

Marketing, not a customer list. A 1999 Asymetrix page said companies like AT&T;, Lucent, Metropolitan Life, and Boeing were doing online learning with the company's tools. "Like" is doing a lot of work. That sentence is not evidence those firms ran ToolBook II 8.1, or any particular version.

The Department of Defense was, in Asymetrix's own telling, one of the largest markets for this kind of training software. The interesting part for this project is narrower. The lessons on disk are one contractor's electronics curriculum for the Navy, built on the native 8.1 runtime, not the DHTML export and not Librarian.

Who would care now

Not a new generation of authors. The product is dead. Assistant was already retired before the end. Nobody needs a pitch for writing next year's course in OpenScript.

The people who still have a reason:

- Anyone holding an 8.x book they cannot open. That is a 16-bit runtime on a 64-bit machine, which does not start. A 32-bit Windows XP virtual machine still runs it. A reader that does not need the virtual machine is the other way through the same door. The Nida electronics set is the case this project is for. The same door fits other native 8.x CBT that was distributed on CD and then stranded.
- A school or a contractor that bought the Nida trainer and still has the discs. The trainer half is the part that talked to Nida's own systems. The lesson half is the books.
- An author who still has the `.tbk` and wants the text, the figures, and the questions out. ToolBook 11.5's `.TBKX` export is the vendor's own answer to that, if the old book opens there. It has not been tried on these lessons. See WV-009.
- Preservation. ToolBook is a 1990 Windows authoring line with a real user conference, a named language, and a body of training material that will not run on a current desktop. That is a software-history problem, not a product problem.

Indifference is the honest default for everyone else. A modern course writer does not gain anything by learning OpenScript. A SumTotal LMS customer in 2026 is not a ToolBook 8.1 user. The ledger should not grow a claim that "the industry wants ToolBook back." It does not.

ToolBook 11.5 and the TBKX export

Recorded 2026-10-12 from correspondence with JohnDoe1.

JohnDoe1 spent 24 years at SumTotal Systems as a ToolBook technical writer, support technician, trainer, and programmer, created the official ToolBook Knowledge Base, and kept hosting it after the product was ended in late 2021. JohnDoe1 also kept an archive of the installers that were shipped, including 8.1 and 11.5.

The personal name, email address, and website are not in this repository.

What JohnDoe1 reported

ToolBook 9.5 and later can store a book as `.TBKX`. That file is a ZIP. Inside it are the images and other media, and an XML description of the book: pages, objects, bounds, layers, and properties.

JohnDoe1 said ToolBook 11.5 has a hidden command that writes this ZIP. If an 8.1 lesson opens in 11.5, the export is a documented structure. The 8.1 on-disk records are not.

Two format notes match what is already known about these files:

- An 8.x book stream begins `03 4A 42 4F`.
- Later ToolBook files use a PE wrapper. The NIDA 8.1 shells already do. The first `0xC00` bytes are a PE stub, and the JBO stream starts at `ZZZTBKBOOKLOC`.

Running 11.5

- 11.5 is 32-bit and runs on 64-bit Windows. ToolBook before 9.0 is 16-bit and does not.
- 11.5 requires MSXML 4. MSXML 6 is not a substitute.
- The last release is the 2012 installer, `TB115.zip`. Patch 4 is applied after install. JohnDoe1 distributes both from the Knowledge Base.
- The first launch starts a fully functional 30-day trial. An activation code is no longer available.
- The trial clock belongs to that Windows installation. The method JohnDoe1 described is a virtual machine: save a clean base image, install 11.5 as the working image, and when the trial ends restore the base and install again.
- A book saved by the trial forever shows "This was created using the Trial Version Of ToolBook" when the runtime opens it. Only a save from an activated copy clears that message. Exporting an existing book was not tested here, so it is not known whether an export counts as that save.

Status on 2026-10-12

JohnDoe1 had found the archival notes for the path from 8.1 through 11.5 to `.TBKX`, and was testing them before sending the steps. The steps were not in hand when this note was written. No NIDA lesson has been opened in 11.5 yet.

The 8.1 work is unchanged. Page markers, the OpenScript subset, and the polygon painter still stand. `.TBKX` is a second way to read the same books, if 11.5 will open them.

When a machine is free, the test is: install 11.5 in a snapshotted virtual machine, open `9e954d01.tbk`, find the hidden save command, and compare the XML with the `80 65` records.

ToolBook II 8.1 module catalog

Recorded 2026-10-11 from version resources and export names on the unpacked images. This is the native path the NIDA lessons use. It is not the DHTML path.

Process and thunk

Module | Role

tbload.exe | Course opener. Reads the Click2learn shell and starts the runtime.

TB80MAIN.EXE | Small PE stub on the runtime side of startup.

TB80RUN.EXE | 32-bit launcher. Exports only mutex wait and release.

TB80RTM.EXE | 16-bit shell. Main window, print, DDE, find, history, status bar. Window classes include EGYPT* and TBKClient.

TB80W32.DLL | 32-bit half. TBXInstance, MEM1632, DIB dither. About 246 exports.

TB80W16.DLL | 16-bit half. CallProc32W, file thunks, palette and graphic entry points. About 406 exports.

TB80F32L.dll | Flat 32-bit entry and the common-dialog hook.

TBFILE32.DLL | Long filenames for the 16-bit engine.

TB80NET.EXE | Network access provider. A 16-bit NE executable, not a DLL.

Book engine

Module | Role

TB80BAS.DLL | Book server. 606 exports, the CDB* API. File description: "ToolBook Server DLL".

TB80CMP.DLL | OpenScript compiler and interpreter. Keyword resources 0x77 and 0x78.

TB80UTL.DLL | Object classification table: Page, Button, Field, Polygon, and the rest.

TB80RCR.DLL | Linked by TB80RTM beside the book server. Exports not fully annotated.

TB80ENU1.DLL | 16-bit English string table.

TB80ENU2.DLL | 32-bit copy of the same strings.

Landmarks: CDBINIT is TB80BAS ordinal 1, CDBFILEOPEN is ordinal 8, CDBFILESAVE is ordinal 9, CDBVIEWPAGE is ordinal 15, CDBVIEWBKGND is ordinal 18. COMPILERCOMPILETEXT is TB80CMP ordinal 1, COMPILEREXPANDTEXT is ordinal 3, CFUNEXECUTE is ordinal 8, CFUNEVALUATE is ordinal 9.

Drawing and media

Module | Role

TB80BMP.DLL | Graphic display, metafile play, FX blit.

TB80JPG.DLL | JPEG. The version resource still carries the old name TB60JPG.DLL.

TB80FLT.DLL | Filter and clip editor.

TB80LNL.DLL | Multimedia ToolBook media engine. The notify callback is Lionel.

TB80MM.DLL | Multimedia notification only. Small.

OLE and old extensions

TB80OLE, TB80OCX, TB80ACX, TB80VBX, and TB80RMT are clipboard, control, and automation support. The course opener does not require TB80RMT. None of these are on the path that paints a Logic Families page.

`linkDLL` libraries add commands only when a handler links them: `TBDLG`, `TBWIN`, `TBDOS`, `TBREG`, `TBDB3`, `TBPDX`, `TBCBT`. `TBCBT` is the CBT scoring extension.

System books

`Book | Role`

`tb80r.sbk | Reader script. Menu and status behavior that is not in TB80RTM.`

`tb80hyp.sbk | Hyperlink and help behavior.`

`tb80anm.sbk | Path animation.`

`nc81base.sbk, nc81test.sbk, nc81recs.sbk, nc81trnr.sbk | NIDA shell: base, test, records, trainer.`

`nc81int.dll, nc81rec.dll | Course interop and the record store. Not Asymetrix.`

A system book adds a script above the book. A `linkDLL` adds commands. Most of what an author treats as ToolBook is one of those two, not a new entry point in `TB80BAS`.

Still thin

`TB80RCR` exports are only partly named. The `TB80UTL` classification indexes are not yet tied to the kind bytes inside `14 00 00` chunks. Author binaries `TB80EDT.DLL` and `TB80.EXE` are not in this set.